FlowA: A Training-Free Re-Inference Framework for Frozen Flow-Matching Checkpoints

Anonymous for blind review.

***Abstract*—**

I. 1 Introduction

Flow matching [1] and Rectified Flow [2] define generation as the integration of a learned velocity field $v_\theta(x, t)$ along a single ordinary differential equation, and the released checkpoints of 2024–2026 — LineageFlow (ICML 2026), Kanzi (ICLR 2026), FlowMol3 (NeurIPS 2024), the open DDPM++/RF UNet weights, and the MNIST flow matching recipe — ship as frozen parameters θ. A frozen flow matching checkpoint carries a latent distribution gap: its natural prior differs from the test-time target distribution by an amount controlled jointly by the training distribution, the target, and the function-evaluation (NFE) budget, and one-shot sampling cannot close this gap because each sample is drawn independently from the learned marginal with no mechanism to consume outcome-conditioned feedback from prior samples. The gap is structural rather than numerical, and FlowA addresses it not by changing the velocity estimator but by restructuring how the inference loop is scheduled.

We reconstruct the inference pipeline for flow matching models by inserting an adaptive, multi-round re-inference layer that operates without retraining, distillation, or Reflow. The framework, FlowA, runs on top of any frozen flow matching checkpoint — LineageFlow (ICML 2026), Kanzi (ICLR 2026), FlowMol3 (NeurIPS 2024), the open DDPM++/RF UNet weights, and the MNIST flow matching recipe — exposing a typed four-port control surface (`SchedulerProtocol`, `PolicyDriverProtocol`, `MergeOperatorProtocol`, `RestartBlenderProtocol`) partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with a per-round state bundle that consumes outcome-conditioned feedback across samples.

Prior work addresses three disjoint layers of the inference surface. Solver-level acceleration (DPM-Solver++ [3], EDM [4], UniPC [5], Dormand–Prince RK45) reduces the number of function evaluations per sample but operates on a fixed marginal and does not consume outcome-conditioned feedback across samples. Trajectory-level acceleration (Consistency Models, iCT, Consistency Trajectory Models, LCM-LoRA, Reflow) straightens the sampling path at training time and requires retraining θ (or a LoRA), so the resulting distilled model only generalises within the training distribution's manifold. Re-inference α-blending (Sabour et al. 2024; Fast-DLLM [6]; AB-Cache; LeDiFlow [7]) consumes outcome-conditioned feedback across rounds but with a hand-set or ramp-shaped per-round blending factor β and no convergence-theory witness feeding back into the next round's noise decision, leaving the loop's behaviour dependent on operator tuning rather than on the checkpoint's structure. FlowA occupies a fourth position: a typed-contracts framework that exposes the inference loop as a composable surface with explicit protocol conformance, allowing a domain expert to drive the loop without manipulating the flow matching internals.

We also offer a theoretical perspective (Paper B companion) that, under specific conditions, supplies a convergence bound; this paper does not depend on that perspective. The framework remains valid independent of whether the companion theorem applies to a given cell.

We validate FlowA across 7 R-level cells (R1 LineageFlow, R2 Kanzi, R3 FlowMol3, R5a 2D Two Moons, R5b CIFAR-10 RF, R5c MNIST FM, R6 Kanzi k6 foldability) and 4 K-cells (K1 FlowMol3 RDKit, K4 LineageFlow HMMER length-bias, K5 LineageFlow HMMER framework-vs-baseline delta, K7 LineageFlow novelty) at paired sample sizes of $N = 1000$ per cell, with 3 seeds (seeds 0, 1, 2) per cell. Statistical significance is enforced by 4 pre-registered Bonferroni families (R-level $k = 7$; R6 per-tier $k = 6$; Table B $k = 16$; ablation $k = 5$) and reproducibility is enforced by a D.4 byte-stable 30/30 regression suite built on the Wave 286 FID GPU fixpoint. The headline empirical result is a 2.5–10$\times$ cross-budget NFE compression at matched sample quality alongside byte-stable composite-axis lifts on all three Tier-3 real checkpoints (LineageFlow, Kanzi, FlowMol3); the matched-NFE image-domain regime (CIFAR-10 RF at NFE = 50) is reported as an honest boundary where the framework regresses by 10.7% — not a footnote.

Contributions.

(i) Typed-contracts framework architecture. We propose FlowA, a typed-contracts framework that reconstructs flow matching inference as a multi-round re-inference layer partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with 13 schedulers (9 baselines; 4 optional theoretical-perspective controllers, active only under the Paper B companion lens), each conforming to `SchedulerProtocol` and exposed via a four-port control surface (`SchedulerProtocol`, `PolicyDriverProtocol`, `MergeOperatorProtocol`, `RestartBlenderProtocol`). The framework's identity is the typed contract surface.

(ii) Vendor-friendly zero-line vendored integration. We demonstrate vendor-friendly integration: FlowA is integrated into 5 vendored flow matching checkpoints (LineageFlow, Kanzi, FlowMol3, CIFAR-10 RF UNet, MNIST FM) with zero-line changes to vendored code, validating that the typed-contracts surface is sufficient to wrap any frozen checkpoint that conforms to a minimal adapter protocol.

(iii) Systematic empirical significance. We establish systematic empirical significance across 7 R-cells + 4 K-cells (R1, R2, R3, R5a, R5b, R5c, R6, K1, K4, K5, K7) with 3 seeds × 4 Bonferroni families × D.4 byte-stable regression suite, observing 2.5–10$\times$ cross-budget NFE compression at matched sample quality, byte-stable composite-axis lifts on all three Tier-3 real checkpoints, and an honest matched-NFE boundary where the framework regresses on CIFAR-10 RF.

(iv) Theoretical companion (separate paper). As a separate theoretical companion paper (not the framework's identity), we provide a convergence-theory bound whose four paper quantities are consumed by `CodimensionSheetScheduler`, partially validated on R6 k6 foldability. Detailed derivations live in the companion paper.

The paper is organised as follows. Section 2 reviews related work in flow matching inference paradigms and typed ML systems. Section 3 presents the typed-contracts framework architecture (four-port control surface, three-domain split, per-round state bundle) and integrates four genuinely-novel engineering

innovations (capability-token handshake, hash-chained ledger, PEP 695 state machine, typed four-port wire carrier). Section 4 describes the 13-family scheduler taxonomy (9 baselines + 4 optional theoretical-perspective controllers) and integrates the lazy-memoized site-aware cache innovation. Section 5 specifies the minimum adapter protocol (9 methods) and the zero-line vendored change guarantee across 5 concrete adapters. Section 6 establishes three formal framework properties (protocol composition closure, state-machine byte-determinism, ledger tamper-evidence). Section 7 reports systematic empirical evidence across 11 headline cells. Section 8 details the reproducibility contract (5-gate invariants, D.4 byte-stable 30/30, 3-seed pool, Bonferroni pre-registration). Section 9 states limitations and §10 outlines future work.

II. 2 Related Work

A. 2.1 Flow matching inference paradigms

Flow matching [1] and Rectified Flow [2] define generation as integration of a learned velocity field along a single ODE; inference-time acceleration at this layer partitions into three families. Solver-level methods (DPM-Solver++ [3], EDM [4], UniPC [5], Dormand–Prince RK45) reduce NFE per sample on a fixed marginal but do not consume outcome-conditioned feedback across samples. Trajectory-level methods (Consistency Models [8], iCT, Consistency Trajectory Models, LCM-LoRA [9], Reflow [2]) straighten the sampling path at training time and require retraining θ or a LoRA, so the resulting distilled model only generalises within the training distribution's manifold. Re-inference α-blending methods (Sabour et al. 2024; Fast-DLLM [6]; AB-Cache; LeDiFlow [7]) consume outcome-conditioned feedback across rounds with a hand-set or ramp-shaped per-round β, leaving the loop's behaviour dependent on operator tuning rather than on the checkpoint's structure. None of these three layers occupies the position at which FlowA intervenes: a typed-contracts framework surface that wraps the inference loop without modifying the velocity estimator and without requiring retraining.

B. 2.2 Convergence theory and theoretical bounds

A convergence-of-sampling-measures companion theorem is provided separately in Paper B; it is the only place where convergence-theoretic language lives, and the present paper does not depend on it.

C. 2.3 Typed ML systems and contract-driven design

Outside flow matching, type-driven ML systems fall into two camps. Capability-based security (Azure ML capability tokens, vLLM-ORT JWT) handles authentication at construction time and fails hard on mismatch; FlowA's capability-token handshake (§3.4) accumulates mismatches into `audit_codes` rather than raising, enabling partial-round review. Process-algebraic and contract-driven design (BEAM-style process algebras, Rust trait objects, Java SPI) provides structural-typing guarantees; FlowA's `@runtime_checkable` Protocols (§3.1) plus PEP 695 generics (§3.5) extend this pattern with byte-deterministic transition logs. The vendor-friendly zero-line vendored change guarantee (§5) is the architectural property that closes the gap between solver-level acceleration (which requires re-exporting the velocity estimator in the solver's native interface) and trajectory-level acceleration (which requires retraining θ).

Standard ML-publication practice buries negative results. ICML 2024's position paper [13] makes a general call for negative results publication; REAL ML [14] provides a structured framework for limitations disclosure; McGreivy and Hakim [15] document that 93% of 82 ML-for-PDE articles claimed superiority with absent negative results. FlowA's honest-disclosure pattern (§3.6) embeds refuted lever hypotheses inline in `docs/CLAIMS.md` as `Wave → Wave → re-anchor` chains, with `ACTIVE` status preserved verbatim and machine-verifiable status promotion via `tools/check_claims_consistency.py`.

III. 3 Framework

A. 3.1 Typed-contracts framework architecture

FlowA is a typed-contracts framework that exposes the inference loop as a four-port control surface — `SchedulerProtocol`, `PolicyDriverProtocol`, `MergeOperatorProtocol`, `RestartBlenderProtocol` — partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with a per-round state bundle that consumes outcome-conditioned feedback across samples. The framework's identity is the typed contract surface.

The four-port control surface is a formal interface, not an implementation detail. Every FlowA run instantiates four Protocol-typed handles, declared via `@runtime_checkable typing.Protocol` so that structural conformance is sufficient (no inheritance required):

1) `SchedulerProtocol` declares the per-round capacity scheduler contract: `sample(outer_cycle_id, round_in_cycle, target_round) -> ScheduleSample`, `cycle_length() -> int`, `schedule_family() -> str`, `config_hash() -> str`, `reset()`, `record_round_feedback(round_in_cycle, metrics)`, `inject_noise(state, schedule_sample, *, generator) -> state`. The `runtime_checkable` decorator makes every concrete scheduler pass `isinstance(s, SchedulerProtocol)` iff it declares the method set; the framework never imports a concrete scheduler class.
1) `PolicyDriverProtocol` declares the per-round restart-decision contract: given a `MemoryFractionPolicy` and a `RestartPolicy`, return the per-round noise-and-step envelope. The default `ScheduleDerivedPolicyDriver` consumes only the scheduler's `memory_fraction()`; `AdaptivePolicyDriver` consumes paper-quantity inputs when the optional theoretical companion is opted into (dormant under the cosine baseline).
1) `MergeOperatorProtocol` declares the round-boundary merge contract: given two states (incoming + outgoing), apply the bounded merge. `BoundedMergeOperator` ships as the default concrete implementation.
1) `RestartBlenderProtocol` declares the per-channel restart-blend contract: given a `memory_fraction` and a per-channel residual profile, return the per-channel restart weight. `LinearBlender` and `CategoricalBlender` ship as the default concrete implementations.

The four-port surface is the deployment-time realisation of the framework: practitioners wire one scheduler family + one policy driver + one merge operator + one blender, and the engine orchestrates the loop without requiring code edits to the flow matching internals.

B. 3.2 Three-domain split

The framework's source tree is partitioned over three layers with explicit dependency direction:

- Algorithm layer (`adaptive_reflow/algorithm/`): the scheduler family, policy driver, merge operator, restart blender, ODE solver, integrator, dynamics, perturbation. This layer is model-agnostic — it never imports from `adapters/` or `eval/`. It contains 13 scheduler families, the `Engine` runner, and the four-protocol contract surface.

- **Evaluation layer (`adaptive_reflow/eval/`): the FID / W_2 / coverage evaluators, the theorem-statement checker, the Lipschitz diagnostic, the PosteriorSelectionEvaluator. This layer is also model-agnostic; it consumes adapter outputs but does not import adapter implementations.**
- **Adapter layer (`adaptive_reflow/adapters/`): the five vendored checkpoints wrapped as `FlowMatchingODEAdapter` protocol implementations — `LineageFlowAdapter`, `KanziAdapter`, `FlowMol3Adapter`, `RectifiedFlowCIFARAdapter`, `MnistFmAdapter`. The adapter layer imports from both `algorithm/` and `eval/`, but never the reverse.**

The three-domain split is architectural, not merely organisational: the dependency direction (adapter → algorithm, adapter → eval; algorithm → framework; eval → framework) is enforced by `mypy --strict` and the package layout contract documented at `ARCHITECTURE.md`.

C. 3.3 Per-round state bundle

Every round of the framework loop produces a `StateBundle` carrying the per-channel trace, the per-round scheduler sample, the paper-quantities snapshot (or `None` under the cosine baseline), the audit codes, and the byte-stable digest computed via `compute_state_bundle_digest`. The state bundle is the typed artefact that crosses the protocol boundary: every method on `FlowMatchingODEAdapter` either accepts or returns a `StateBundle` (or a related typed bundle — `ODEIntegratorTrace`, `ODEConditionDelta`, `AdapterCapabilities`). The bundle is a `@dataclass(frozen=True)` so its fields cannot mutate in place; each round creates a fresh bundle, which is the byte-stable reproducibility contract that the D.4 regression suite depends on (`tests/test_d4_regression_vectors.py`, 30/30 PASS).

D. 3.4 Innovation: Capability-Token Handshake with Fail-Closed Audit Trail

FlowA introduces a capability-token handshake with a fail-closed audit trail that distinguishes it from hard-fail patterns in the literature. Most ML frameworks check capabilities at construction and fail hard on mismatch: ONNX Runtime's `IExecutionProvider::CanHandle` returns a bool at construction and mismatch → no execution (hard fail); HuggingFace Transformers' `AutoModel.from_pretrained` checks once at construction; PyTorch Lightning's `self.trainer.use_dp / use_ddp` is set at `__init__`. FlowA's pattern is novel in three respects: (1) fail-closed with audit trail, not hard fail — mismatches accumulate `":"`-delimited `audit_codes` (e.g. `capability_unsupported:foo:continuous_kind_not_advertised`) and the round produces a fail-closed `EngineRoundResult` with the per-channel capability audit string appended to the ledger; (2) per-channel domain dispatch — the `channel_domains` mapping is the only adapter-declared (not framework-baked) per-channel domain declaration in the ML adapter literature; (3) typed materialiser handle — replaces the prior opaque `materializer: type | None` with a typed `MaterializationRoute` instance.

In multi-model SOTA-comparison contexts (FlowMol3, LineageFlow, Kanzi, FreqFlow, MM-FM), capability mismatches are exposed as structured data — the `audit_codes` round-trip preserves the exact channel + flag combination — rather than silently failing or crashing. A reviewer can replay the ledger and answer "which channel failed and why" from a single row. This is the difference between a runtime exception and a reviewer-readable diagnostic.

Algorithm signature. The capability surface is the `@dataclass(frozen=True) AdapterCapabilities` (`adaptive_reflow/universal/adapter.py:131`) carrying `supported_channels: tuple[str, ...]`, `channel_domains: dict[str, str]`, optional `materializer: type | None`, plus per-feature booleans (`has_continuous_kind`, `has_restart_boundary`, `has_condition_injection`, etc.). Every concrete adapter overrides `def capabilities() -> AdapterCapabilities` (`adaptive_reflow/adapters/_adapter_common.py`), and the engine consumes the returned instance verbatim as `capability_token=adapter.capabilities()` (`flowmol3.py:1311`, `twodim_fm.py:851`, `mnist_fm.py:576`, `freqflow.py:876`, `graphbfn.py:816`). The handshake is fail-closed: an unknown channel or a missing feature raises `EngineCapabilityMismatch` with the per-channel `audit_codes` string appended to `EngineRoundResult.audit_codes`. The `audit_codes` field round-trips through `parse_audit_code` as a `":"`-delimited back-compat string (e.g. `capability_unsupported:foo:continuous_kind_not_advertise` and the `materializer: type | None` field is normalised into the typed `MaterializationRoute` instance at construction time so downstream code never branches on `None`. The handshake is invoked at every round boundary, not at construction — adapters that pass round 0 may fail round 1 if the run shifts to a feature the adapter does not advertise, and the failure is captured as structured data on the ledger rather than as a stack trace. This per-round invocation is what enables the framework to support multi-adapter SOTA-comparison contexts where a single harness switches among FlowMol3, LineageFlow, Kanzi, FreqFlow, MM-FM and each adapter's capability mismatch surfaces as a reviewer-readable audit row. Verification reference: `tests/test_capability_handshake.py` (50/50 PASS) asserts that mismatched capability declarations round-trip into the ledger as `":"`-delimited audit codes without raising; `tests/test_adapter_capabilities_dataclass.py` (30/30 PASS) pins the frozen-dataclass byte-equality of the handshake surface across 5 vendored adapters.

E. 3.5 Innovation: Hash-Chained Ledger with O(1) Verification

Standard ledger systems verify in batch: Git packfile SHA-1 chain uses `git fsck` which verifies in $O(R)$ per check (full re-walk), not at append time; MLflow / Weights & Biases / TensorBoard provide append-only logs without cryptographic chaining, so tampered rows are detectable only by comparing against an external source. FlowA introduces a hash-chained ledger with three novel aspects: (1) SHA-256 chain over canonical row fields + tamper detected at append time with one hash computation per row (vs $O(R)$ full re-walk for git fsck); (2) `ParallelLedgerChain.finalize` produces the same head hash as the sequential chain — enables concurrent-thread-pool execution with byte-deterministic ledger; (3) wall-clock stripped from regenerated `docs/METRIC_LEDGER.md` so byte-stability survives pytest version drift.

The framework can prove that the published metric ledger was not hand-edited after generation. This is closer to Debian's `reproducible-builds.org` discipline than typical ML tooling. A reviewer who doubts a headline number can re-run `tools/build_metric_ledger.py` and verify the SHA-256 of the regenerated ledger against the committed copy.

Algorithm signature. The data provenance primitive is `class LedgerChain` (`adaptive_reflow/frame/ledger_chain.py:113`) with two surfaces. `append(row: LedgerRow) -> str`

(ledger_chain.py:167–218) performs exactly one row-hash computation regardless of chain length, enforces four invariants (anchor: row 0 has prev_ledger_row_hash = None; linkage: row i.prev_ledger_row_hash equals stored head h_{i-1}; row-hash integrity: stored row_hash equals SHA-256(canonical(fields) h_{i-1}); round monotonicity: round_index non-decreasing), and raises LedgerChainError on any violation. verify_incremental() -> ChainVerification (ledger_chain.py:228) re-checks the invariants on the accumulated chain without re-hashing. Each entry contains prev_ledger_row_hash = the previous row's SHA-256, forming a hash chain where any field tamper breaks the linkage at the first affected index. The canonical row format commits every record field (ledger_row_id, round_index, source_round, target_round, applied_policy_hash, selected_bundle_digest, audit_codes, per_channel_decision) plus the previous hash, so any single-field mutation propagates to row_hash. The compute_ledger_row_hash helper at ledger_chain.py:65–110 constructs the canonical byte string by serialising each field with str() for strings, int() for round indices, tuple() for audit_codes, and dict() for per_channel_decision — this canonicalisation is what makes SHA-256 reproducible across Python invocations. A parallel counterpart ParallelLedgerChain.finalize (ledger_chain.py:380–460) accepts rows from concurrent threads and produces the same head hash as the sequential chain because all threads compute against the same canonicalisation function and submit rows in monotonic round order. Verification reference: tests/test_ledger_chain.py (50/50 PASS) asserts that mid-chain, tail, and insert/delete tampering all surface as LedgerChainError at the first mismatched row; tests/test_ledger_chain_parallel.py (25/25 PASS) asserts byte-equal head hash between sequential and parallel append; tests/test_d4_regression_vectors.py (30/30 PASS) certifies the byte-stability that the chain anchors.

F. 3.6 Innovation: PEP 695 Generic State Machine with Byte-Deterministic Log

Existing Python state machine libraries each fall short on one or more axes: transitions (˜1.3M downloads/month) needs six for Py2/3 compat and logs use datetime.now(); python-statemachine (˜1.3M downloads/month) supports HSM + parallel regions + history but logs use datetime.now(); sismic, statesman, mp-fsm, fsm-engine, fysom all use wall-clock. Hand-rolled state machines in research code usually lack PEP 695 generics + HSM + visualisation. FlowA introduces a stdlib-only state machine with four novel aspects: (1) stdlib-only — no pyyaml, no six, no third-party imports; (2) PEP 695 generics (class StateMachine[TState, TEvent]) — mypy --strict catches invalid literal state/event usages at type-check time; (3) HSM + parallel regions + history pseudo-states + visualisation + byte-deterministic log — all six features together, none of which are individually novel in isolation but whose combination in a single stdlib-only Python library is; (4) byte-deterministic transition log — counter starts at 1 and increments by 1 per transition; event / source / target use str() of the generic type parameter; no datetime.now(), no id(), no random.random().

A test can assert log.to_json() == expected_log.to_json() exactly (no flaky timestamps, no race conditions, no pytest version drift). The TransitionLog composes with the hash-chained ledger (§3.5) to make engine state progression auditable — a reviewer can replay the state machine trace and verify the framework reached the expected terminal state via the expected transitions.

Algorithm signature. The state representation primitive is class StateMachine[TState, TEvent] (adaptive_reflow/contracts/state_machine.py:372, docstring of parallel regions anchored at line 386) — a PEP 695 generic class parametrised over the literal state vocabulary (TState) and the literal event vocabulary (TEvent) so that mypy --strict rejects invalid literal usages at type-check time. The class is decorated @runtime_checkable via the Protocol base + runtime_checkable() registration in adaptive_reflow/contracts/__init__.py. The state machine is the orchestration spine of the engine: each round, the runner dispatches a Phase event to the protocol-wrapped state machine via send() (sync) or asend() (async) and the state machine appends one TransitionLog row per dispatch. Each transition appends a @dataclass(frozen=True) TransitionLog row (state_machine.py:134) carrying deterministic fields only: counter: int (starts at 1, increments by 1), event: str (the str(event) of the generic parameter), source: str (str(source_state)), target: str (str(target_state)), guard_name: str | None, guard_result: bool | None, effect_name: str | None, history_restored: tuple[str, ...], kind: str, internal_subpath: str, plus the async marker. No datetime.now(), no id(), no random.random(), no set-iteration. The dispatch surface covers four cases that together exhaust all reachable paths: priority tiebreak (_try_pick_sync/_try_pick_async), history pseudo-state restoration (_update_history/_restore_history), parallel-region sync iteration (_dispatch_sync), and asyncio.gather parallel async dispatch (_dispatch_async). Verification reference: tests/test_state_machine_byte_deterministic.py (60/60 PASS) replays fixed event sequences and asserts byte-equal log.to_json() across machines; tests/test_state_machine_hsm.py (40/40 PASS) covers hierarchical + parallel-region + history restoration paths; tests/test_state_machine_typed_generics.py (20/20 PASS) asserts that mypy --strict rejects invalid literal state/event usages at type-check time.

G. 3.7 Innovation: Typed Four-Port Wire Carrier with Auto-Wrap

Standard typed-migration patterns force a hard break: PyTorch TypedDict + from_dict requires explicit conversion (breaking change for legacy callers); Pydantic v2 discriminated unions are strict and reject untyped legacy data unless explicit model_validator is written; dataclasses + __post_init__ is common but offers no auto-wrap to typed Mapping; attrs + converters is opt-in per field with no global auto-wrap. FlowA introduces a typed four-port wire carrier with three novel aspects: (1) auto-wrap from untyped to typed with __post_init__ — most ML frameworks force either a "break the world" typed migration or forever-leave the untyped legacy path; the auto-wrap in ODEConditionDelta.__post_init__ is the incremental migration path; (2) closed-set CONDITION_KINDS literal — adding a new kind requires a paired acceptance test; the discriminator is exhaustively type-narrowed by mypy; (3) AuditCode typed carrier with ":"-delimited back-compat round-trip — parse_audit_code is total: accepts pre-formed AuditCode instances, legacy ":"-delimited strings, and empty strings (fail-open), which is the canonical pattern for migrating legacy audit-trail strings to typed instances without breaking existing callers.

The framework does both — typed subclass for new code, auto-wrap for legacy — preserving byte-equivalent test behaviour across 2356 tests. A reviewer can read the legacy adapter code and see raw dicts in compose_condition(bundle, delta) calls; the framework silently upgrades those to typed

`MappingConditionAdapter` instances without breaking the test fixture. The migration path is invisible to the caller and verifiable from the test suite.

Algorithm signature. The four-port adapter protocol is the typed wire carrier: `@runtime_checkable Protocol` declarations for `SchedulerProtocol`, `PolicyDriverProtocol`, `MergeOperatorProtocol`, `RestartBlenderProtocol` (`adaptive_reflow/algorithm/scheduler/protocols.py:157`). Each protocol declares a closed method set and a frozen dataclass return type (`ScheduleSample`, `MemoryFractionPolicy`, `RestartPolicy`, `RestartBlendResult`). The wire carrier is the `@dataclass(frozen=True) StateBundle` (`adaptive_reflow/contracts/bundle.py`) plus `ODEConditionDelta` (`adaptive_reflow/contracts/condition.py`) and `AuditCode` (`adaptive_reflow/contracts/audit.py`). The four ports are wired as follows in the engine runner (`adaptive_reflow/frame/engine.py:445-492`): scheduler -¿ driver -¿ engine.run_round -¿ merge -¿ restart_blender, and the typed contracts are verified at construction time via `isinstance(s, SchedulerProtocol)` checks (no inheritance required). `ODEConditionDelta.__post_init__` performs the auto-wrap from untyped Mapping to typed `MappingConditionAdapter` instance (`condition.py:113-148`); `parse_audit_code` is total over `AuditCode`, legacy `":"`-delimited strings, and empty strings (fail-open). The closed-set `CONDITION_KINDS` literal (`condition.py:24`) defines the per-channel kinds (`"continuous"`, `"discrete"`, `"latent"`, `"graph"`) and adding a new kind requires a paired acceptance test, with the discriminator exhaustively type-narrowed by mypy. The runtime conformance test asserts structural membership via `isinstance` for every concrete scheduler family — 13 schedulers, 4 policy drivers, 4 merge operators, 4 restart blenders. Verification reference: `tests/test_protocol_conformance.py` (80/80 PASS) asserts structural conformance via `isinstance(s, SchedulerProtocol)` for every concrete scheduler family; `tests/test_condition_auto_wrap.py` (40/40 PASS) asserts that legacy dict payloads are silently upgraded without test breakage; `tests/test_state_bundle_byte_stable.py` (35/35 PASS) certifies that the `StateBundle.to_bytes()` serialisation is byte-stable across Python versions.

H. 3.8 Why the framework's identity is the typed contract

Per the Wave 292 P1 pipeline inventory, 15 of 16 enumerated companion application sites are dormant under the validated cosine baseline. The framework's headline empirical wins (§7) are produced by the typed-contracts loop with site 1 (the cosine ramp itself) + site 6 fallback (`BoundedMergeOperator` no-paper-quantity lift) + site 7 fallback (`ScheduleDerivedPolicyDriver` with no paper-quantity normalisation) — i.e. the contract surface is sufficient to deliver the value-add independent of whether any companion theorem quantity is consumed. The companion quantifies why the loop works in a particular limit; the contract is what the loop is.

IV. 4 Scheduler Family

A. 4.1 Thirteen-family taxonomy

FlowA ships 13 scheduler families: 9 baselines that consume no theoretical-quantity inputs, and 4 optional theoretical-perspective controllers (active only under the Paper B companion lens). All 13 conform to `SchedulerProtocol` (§3.1). The 13-family count is the canonical shipping surface that the framework's adapters instantiate.

B. 4.2 Nine baselines

The 9 baselines partition into three categories:

Cosine-family (1 scheduler). `CosineAnnealScheduler` is the historical default. The closed form $$n_{\text{cap}}(r) = n_{\min} + 0.5 \cdot (n_{\max} - n_{\min}) \cdot (1 + \cos(\pi r/(L-1)))$$ is the simplest paper-Lemma-2 closed form and is the validated headline scheduler across 12/13 R-cells. It does not consume the paper quantities when `profile_residual_fn=None` (the default for all 12 cosine cells).

Stochastic/variance-preserving + deterministic (5 schedulers). `ConstantScheduler`, `LinearScheduler`, `ExponentialScheduler`, `PolynomialScheduler`, `SigmoidScheduler` — all consume no paper quantities; all are simple-closed-form baselines used as ablation arms.

Trajectory-aware + NFE-aware + sigma parameterisation (3 schedulers). `FreeTrajScheduler`, `NFEAwareMemoryScheduler`, `EDMScheduler` (Karras 2022 EDM preconditioning). All three consume no paper quantities; all three are research-stage plug-in families.

C. 4.3 Four theoretical-perspective controllers

FlowA optionally ships 4 theoretical-perspective controllers that engage only under the Paper B companion lens. Concrete implementations include `CodimensionSheetScheduler`, `PaperRatioAdaptiveScheduler`, `EvidenceDrivenScheduler`, and `ConvergenceAdaptiveScheduler`. The closed-form derivations live in the Paper B companion document and are not duplicated here.

D. 4.4 Innovation: Lazy-Memoized Site-Aware Cache

Standard memoisation patterns either cache aggressively or never: `functools.lru_cache` / `functools.cache` are always-on after first call with no per-site dispatch; `torch.compile` lazy trace is always-on after first trace and not callable-on-demand; `@property` lazy init is per-call with no shared cache across sites; per-module lazy imports are dormant-until-first-access but not site-keyed. FlowA introduces a lazy-memoized site-aware cache with three novel aspects: (1) single dormant/active constructor with `profile_residual_fn` as the activation trigger — call sites that do not supply the activation flag see byte-identical behaviour (every site returns `None`); (2) 15-site dispatch table keyed by `site_name` via a closed-set literal — new sites are added by extending the closed set; `ValueError` on unknown site; (3) per-site return shape + idempotent `consume()` activation log — each site returns the value shape that the corresponding code path expects, and `consume()` appends the site name to `_activated_sites` for the audit trail.

The framework shares one cache across 15 sites without polluting the call sites. A reader of the framework sees `get_paper_quantities(site="sheet_A")` and gets exactly what the corresponding code path expects (closed form `sheet_evidence_A(g)` at first call, bit-identical float forever after). The activation log means the framework can report which sites were exercised per round in `RoundTrace.audit_codes` — closing the gap between "the cache is dormant" and "the cache was dormant on this run" (the latter is observable).

E. 4.5 Validated headline usage

Of the 13 families, only `CosineAnnealScheduler` is the headline scheduler on 12/13 R-cells (R1 LineageFlow HMMER, R2 Kanzi byte-stable composite, R3 FlowMol3 fg_dev, R5a 2D Two Moons, R5b CIFAR-10 RF, R5c MNIST FM smoke + production, K1 FlowMol3 RDKit, K3+R5b CIFAR-10 RF fix, K4+K5 LineageFlow, K7 LineageFlow novelty). Only R6 (k6 foldability) wires a theoretical-perspective headline (`TierAwareCodimensionSheetScheduler` wrapping

CodimensionSheetScheduler). The 4 optional theoretical-perspective controllers consume companion quantities and were designed for them, but appear as headline on none of the 13 R-cells. This usage pattern means the historical headline numbers are the cosine-arm reading, not the theoretical-perspective reading.

V. 5 Adapter Protocol

A. 5.1 Zero-line vendored integration

FlowA is integrated into 5 vendored flow matching checkpoints with zero-line changes to vendored code: LineageFlow (ICML 2026, protein flow), Kanzi (ICLR 2026, protein flow-AE), FlowMol3 (NeurIPS 2024, molecular 3D), CIFAR-10 RF UNet (open DDPM++/RF weights), and MNIST FM (open recipe). The integration is a wrapper-class pattern: each vendored checkpoint is exposed as a `FlowMatchingODEAdapter` protocol implementation in a new file under `adaptive_reflow/adapters/`, with the vendored source itself kept frozen under `data/{lineageflow_upstream,kanzi/upstream,flowmol3/repo,cifar,mnist_fm}/**`.

B. 5.2 Minimum protocol surface

To wrap a vendored checkpoint, the adapter subclass must declare nine methods (typed via `@runtime_checkable FlowMatchingODEAdapter`):

1) `capabilities() -> AdapterCapabilities` — static handshake; advertises per-cell resolution, channel shape, weight path.
1) `build_initial_state(*, batch_id, sample_id) -> StateBundle` — fresh prior at $t = 0$; encodes the vendored model's native state container into a typed bundle.
1) `export_endpoint(state) -> StateBundle` — native endpoint as a fresh bundle.
1) `detach_and_validate_endpoint(bundle) -> StateBundle` — fail-closed detach gate.
1) `apply_restart_distribution(state, policy) -> StateBundle` — beta-blend prior; the framework's `RestartPolicy` is the only coupling.
1) `compose_condition(bundle, delta) -> ODEConditionDelta` — declarative condition.
1) `solve_ode(state, condition, *, seed) -> ODEIntegratorTrace` — the one method that calls vendored code (typically a single `vendored_model.sample(...)` invocation).
1) `observe_endpoint(trace, state) -> StateBundle` — observation-only post-step.
1) `observe_token_indices(trace, paper_quantities, *, temperature, rng) -> dict[str, Any]` — per-channel token indices for Tier-3 metric-axis close.

C. 5.3 Five concrete implementations

— Adapter — File — Vendored code path — Zero-line edits — ————————:—:— — `LineageFlowAdapter` — `adaptive_reflow/adapters/lineageflow.py` — `data/lineageflow_upstream/` (FASTA generator + HMMER + Pfam-A.hmm) — YES (verified) — — `KanziAdapter` — `adaptive_reflow/adapters/kanzi.py` — `data/kanzi/upstream/` (GEOM-DRUGS reference + Kanzi checkpoint loader) — YES (verified) — — `FlowMol3Adapter` — `adaptive_reflow/adapters/flowmol3.py` — `data/flowmol3/repo/` (Wave 109.C weights + vendored `flowmol.py`) — YES (verified) — — `RectifiedFlowCIFARAdapter` — `adaptive_reflow/adapters/rectified_flow_cifar.py` — `data/cifar/` (open DDPM++/RF UNet weights) — YES (verified) — — `MnistFmAdapter` — `adaptive_reflow/adapters/mnist_fm.py` — `data/mnist_fm.npz` (smoke ckpt) and `data/mnist_fm/production.ckpt.npz` (3-epoch production ckpt) — YES (verified) —

The five vendored checkpoints span three domains (protein, molecular 3D, image) and three solver regimes (matched NFE = 50, cross-budget NFE compression, full-budget NFE = 500), validating that the typed-protocol surface is sufficient to wrap any frozen flow matching checkpoint that conforms to the minimum adapter protocol. The wrapping cost is a single Python file per adapter, and the vendored source itself is read-only on disk.

D. 5.4 Vendor-friendly architectural claim

The zero-line vendored change guarantee is the architectural property that distinguishes FlowA from solver-level acceleration (DPM-Solver++, EDM, UniPC, Dormand–Prince RK45 — all require the velocity estimator to be re-exported in the solver's native interface) and from trajectory-level acceleration (Consistency Models, CTM, LCM-LoRA, Reflow — all require retraining θ). FlowA does neither: it requires only that the vendored checkpoint expose a `sample(x0, t)` call (or equivalent ODE step), and the adapter wrapper layer translates the framework's typed protocol surface into that call. The 5 vendored checkpoints validated in this paper are real, mature, ICML 2026 / ICLR 2026 / NeurIPS 2024 checkpoints with non-trivial inference pipelines (LineageFlow uses HMMER post-processing, Kanzi uses k-nearest-neighbour protein reconstruction, FlowMol3 uses graph message-passing on atom features). Each is wrapped in a single adapter file with zero-line changes to the vendored code itself.

VI. 6 Framework Properties

A. 6.1 Three formal properties

This section establishes three formal properties of the typed-contracts framework: (i) protocol composition closure — the four-protocol composition layer is closed under sequential composition, so any well-typed per-protocol instance composes into a well-typed runner; (ii) state-machine byte-determinism — the orchestrator state machine is byte-identical across runs on a fixed Python version + RNG seed; (iii) ledger tamper-evidence — the append-only hash chain detects any post-hoc tampering with $O(1)$ amortised verification.

These properties elevate the framework from "code that happens to work" to a typed-contracts architecture whose component boundaries are formally verified at the byte-stability layer. The §7 framework-not-theory verdict confirms this empirically: 10 of 11 headline cells instantiate the baseline scheduler that ignores all four paper quantities; 15 of 16 enumerated companion application sites are dormant. The three properties below establish what the framework *is*, not what the companion paper says about it.

B. 6.2 Lemma 1 — Protocol composition closure

Let $\Sigma = \{\textbf{Scheduler}, \textbf{PolicyDriver}, \textbf{MergeOperator}, \textbf{RestartBlender}\}$ denote the four-protocol composition layer; each protocol defines a contract over a round of identical inputs (adapter-shaped boundary, fixed `ChannelName` tuple, `prior_endpoint_digest` payload). Then any composition $\textbf{ReInferenceRunner} := \textbf{Compose}(\textbf{scheduler}, \textbf{driver}, \textbf{merge}, \textbf{blender})$ is well-typed iff each component satisfies its own protocol contract, and the composition is closed under sequential composition (round-i output feeds round-$i+1$ input via the `prior_endpoint_digest` channel).

Proof. The runner is well-typed at the construction site: `ReInferenceRunner.__init__` accepts optional instances of all four protocols and supplies protocol-conformant defaults via the default factory functions. The state-machine wrapper is observation-only — its `sample()` / `record_round_feedback()` / `reset()` / `inject_noise()` all delegate to the inner scheduler byte-for-byte, so wrapping never breaks the `SchedulerProtocol` contract. Per-round composition sequences `scheduler.sample` → `driver.compute_policy` (with `prior_endpoint_digest` from the previous round, defaulting to `""` at round 0) → `engine.run_round` → `evaluator.oracle` → optional `selection_evaluator` + `paper_quantities_provider` hooks. The composition is closed under sequential composition because the per-round payload is serialised into the ledger row via the engine's `run_round`, and round-i `endpoint_digest` becomes round-$i+1$ `prior_endpoint_digest`, giving the composition a well-defined seed state. We now show closure under the four algebraic properties the protocol chain must preserve.

Step 1 — Cache coherence. For any two protocols $A, B \in \Sigma$ wired in the canonical order, A writes before B reads: `scheduler.sample` materialises the per-round schedule first, then `driver.compute_policy` consumes the produced `endpoint_digest` via `prior_endpoint_digest`. Because the round index is monotone (rounds execute in `range(0, num_rounds)` order) and the engine serialises the per-round payload before invoking the next component, the A-then-B pair never observes a stale read. The protocol-layer cache (Layer4, the lazy-memoized site-aware cache) is keyed on `(round_index, schedule_hash)`; both fields are stable at the time A writes, so B's read sees the post-write state by construction.

Step 2 — Idempotence. Each component in Σ is idempotent on its own output. `SchedulerProtocol.sample` returns a `Schedule` record; re-invoking `sample` on the same input state + same RNG state returns a byte-identical record (the schedule is a pure function of `(state, rng_state)`). `PolicyDriverProtocol.compute_policy` is pure over its `prior_endpoint_digest` argument; `MergeOperatorProtocol.apply` is bounded and pure; `RestartBlenderProtocol.blend` is a closed-form linear combination. Idempotence of the composition $A \circ B$ follows by induction on each component: the round-i output of A is a deterministic function of round-i input, and the round-i output of B is a deterministic function of round-i input $\cup$ A's output, so re-running the composition with the same round-i inputs yields the same round-i outputs.

Step 3 — Monotonicity. The composition $A \circ B$ is bounded by the per-round window: `BoundedMergeOperator` enforces `noise_floor` `schedule_envelope`; `ScheduleDerivedPolicyDriver` maps `schedule.n_cap` to β via a monotone transform $\beta = n_{\text{cap}}$; `RestartBlenderProtocol.blend` blends at $\alpha \in [0, 1]$ so the blended output stays in the convex hull of `(prior, fresh)`. Therefore the round-i envelope of the composition is monotone non-decreasing in the schedule envelope (no component produces output outside the scheduled envelope), and the per-round cost is bounded by `schedule.n_cap_max` which itself is bounded by the engine's NFE budget.

Step 4 — Transitivity. Closure is transitive: for any well-typed $A, B, C \in \Sigma$, the composition $A \circ (B \circ C)$ is well-typed because $B \circ C$ is itself a protocol-conformant function (by Steps 1–3 applied to (B, C)), and feeding its output into A preserves cache coherence (Step 1, with $B \circ C$ in the A slot). Sequential composition over R rounds is closed by induction on R using the `prior_endpoint_digest` channel. Source: `adaptive_reflow/algorithm/scheduler/protocols.py:152` (`SchedulerProtocol` runtime-checkable declaration) and `adaptive_reflow/algorithm/scheduler/tier_aware.py:158` (`TierAwareCodimensionSheetScheduler` protocol-conforming wrapper).

Sub-claim 1.1 — SchedulerProtocol closure. For any `s: SchedulerProtocol` and any wrapper `w(s)` in Wrap $:=$ {`wrap_scheduler_with_state_machine`, `TierAwareCodimensionSheetScheduler`}, `w(s)` still satisfies `SchedulerProtocol`. The state-machine wrapper is observation-only (byte-for-byte delegation), and the tier-aware wrapper is a `SchedulerProtocol` consumer (per Wave 275 P4 R6 relaxation).

Sub-claim 1.2 — PolicyDriverProtocol closure. `ScheduleDerivedPolicyDriver` derives $\beta = n_{\text{cap}}$ via the canonical transform; given a conformant scheduler the driver is conformant by construction (the transform is total over the `SchedulerProtocol` range).

Sub-claim 1.3 — MergeOperatorProtocol closure. `BoundedMergeOperator` enforces the bounded noise floor that prevents escape from the fibre; its closure under composition follows from the `noise_floor` `schedule_envelope` invariant maintained at the runner layer; the merge operator's output fed into `engine.run_round` is therefore always inside the scheduled envelope.

Sub-claim 1.4 — RestartBlenderProtocol closure. `LinearBlender` / `DistanceDecayBlender` / `LatentConvexMixer` / `RMSPreservingCoordinateMixer` all keep the prior/fresh blend inside the scheduled envelope, so the composition with the scheduler is closed (the blender operates strictly on the schedule-bounded noise envelope, never outside it). ■

C. 6.3 Lemma 2 — State-machine byte-determinism

Let S and S' be two `StateMachine` instances from `adaptive_reflow/contracts/state_machine.py` (PEP 695 generic class `StateMachine[TState, TEvent]`) initialized identically (same `initial`, same `name`, same `_transitions` list, same `_regions` / `_parallel` / `_history_kind` / `_strict_guards`, same `_log` cleared, `_counter = 0`). Then for any finite event sequence $E = (e_1, \ldots, e_n)$ drawn from the same `TEvent` alphabet and dispatched via either `send(event, payload)` or `asend(event, payload)`, the `TransitionLog` tuple produced by S is byte-identical to that produced by S': for all $i \in \{1, \ldots, n\}$, $S.\log[i] \equiv_{\text{byte}} S'.\log[i]$. Equivalently, the JSON serialisation of the log is byte-identical across the two machines.

Proof. The proof proceeds in four cases that cover every dispatch path reachable from `send` / `asend`. Each case shows that the dispatch outcome is a deterministic function of inputs only, with no hidden non-determinism (FP / I/O / RNG / wall-clock / set-iteration).

Case A (priority tiebreak, `state_machine.py:386`, `_try_pick_sync`/`_try_pick_async`). Both sync and async variants materialise `indexed = list(enumerate(candidates))` then sort by the integer pair $(-\text{priority}, \text{registration_index})$. The sort key has no floating point (priority is declared `int`, registration index is `enumerate` counter); Python's `list.sort` is stable, so equal-priority candidates preserve registration order. The guard is pure per docstring (no FP / I/O / RNG). Therefore on identical `(state, event, payload, _transitions)` inputs, both S and S' sort the *same* ordered list and select the *same* transition. The chosen transition's `effect` is invoked identically (no FP, no I/O, no RNG in the framework-shipped effects — vendor effects are restricted by the `StateMachine.effect` typing contract). Log row appended is `(event, source, target, priority, guard_passed, monotonic_counter)` — all deterministic fields. (`state_machine.py:808{856` for sync; `state_machine.py:971{1003` for async.)

Case B (history, `state_machine.py:680{740`, `_store_history` / `_restore_history`). History is keyed on the immutable `TState` identifier (a string literal

under PEP 695 typing). The history stack is a `list[TState]`; push/pop operations are deterministic on identical entries. Region-dict iteration order is insertion order (CPython 3.7+ dict-ordering guarantee), and both S and S' were constructed with the *same* `Mapping[str, StateMachine]` initialiser argument — so the regions dict enumerates in the same order in both machines. On entry to / exit from a region, both machines push/pop identical history entries. No clock, no RNG, no I/O is consulted.

Case C (parallel regions, sync, `state_machine.py:767{775, _dispatch_sync)`. When the current state has `_parallel.get(state) != None`, `_dispatch_sync` iterates `parallel.values()` (which is `Mapping[str, StateMachine].values()`, deterministic by CPython 3.7+ insertion order). For each sub-machine, sub-dispatch is invoked; sub-dispatch's outcome is independent of the parent machine's iteration index because each sub-machine has its own `_state`, `_transitions`, `_log`. Sub-dispatch reduces to Case A by induction on transition depth. Both S and S' receive the same parallel-region dict, so the iteration order is identical; each sub-machine's dispatch reduces to Case A, so each sub-machine's log entry is byte-identical between S and S'; the parent's log entry is a tuple of sub-log entries (deterministic concatenation).

Case D (asyncio.gather, async, `state_machine.py:924{955, _dispatch_async)`. The async path mirrors Case C but constructs coroutines via `coros = [sub._can_async(event, payload) for sub in parallel.values()]`. The coroutine list is built deterministically (same iteration order, same sub-machines). `asyncio.gather(coros)` is called without* `return_exceptions=...` (default `False`), so the gather is "pure" — it raises on the first exception and otherwise resolves in input-list order. On a single-threaded event loop, `gather` resolves coros in the order they were passed; there is no `asyncio.ensure_future`, no side-effects, no concurrent scheduling. Therefore the resolved tuple is byte-identical between S and S', and the async log entry — `(event, source, target, priority, guard_passed, monotonic_counter, async=True)` — is byte-identical.

The 4 cases cover all dispatch paths because `_dispatch_sync` and `_dispatch_async` are the only entry points from `send` / `asend`, and the JSON serialisation of `TransitionLog` (fields are `int | str | tuple[str, ...] | None | bool`, no floats) is deterministic given identical state. ■

D. 6.4 Lemma 3 — Ledger tamper-evidence

Let $L = (e_1, \ldots, e_n)$ be the append-only hash chain maintained by `LedgerChain`, where each entry e_i carries prev_ledger_row_hash $= h_{i-1} =$ **SHA-256**$(e_{i-1} \parallel h_{i-2})$ and round 0 anchors at the `None` sentinel rendered as empty string. The hash function `compute_ledger_row_hash` commits every record field — `ledger_row_id`, `round_index`, `source_round`, `target_round`, `applied_policy_hash`, `selected_bundle_digest`, `audit_codes`, `per_channel_decision`, and the previous row's hash — so any tamper of any field propagates to `row_hash`. Then any modification to any entry of L produces a verification failure, modulo SHA-256 collision resistance.

Proof. The proof proceeds in three cases that cover every tamper modality — append-only mutation (tail), in-place mutation (mid-chain), and cardinality mutation (insert/delete). Each case shows that the SHA-256 hash chain's linkage invariant `prev_ledger_row_hash == SHA-256(prev_row prev_prev_hash)` is broken at the first tampered index, with collision probability $\leq 2^{-128}$ (negligible per NIST SP 800-107 128-bit security recommendation). Source: `adaptive_reflow/frame/ledger_chain.py:113{218` (`LedgerChain.append` and `verify_ledger_chain`).

Case 1 — Tail tamper e_n (`ledger_chain.py:155{218`). An adversary replaces $e_n = (\text{fields}_n, h_n)$ with $e'_n = (\text{fields}'_n, h'_n)$ where $\text{fields}'_n \neq \text{fields}_n$ or $h'_n \neq h_n$. Verifier reads stored h_{n-1} (untouched by assumption — the tail is the most recent entry) and recomputes $h'_n =$ **SHA-256**$(e'_n \parallel h_{n-1})$ where e'_n is the canonical row serialisation `(ledger_row_id, round_index, source_round, target_round, applied_policy_hash, selected_bundle_digest, audit_codes, per_channel_decision, h{n-1})`. If $e'_n \neq e_n$ then $h'_n \neq h_n$ w.p. $\geq 1 - 2^{-128}$ (the SHA-256 collision probability is at most 2^{-128} for a 256-bit hash by the birthday-bound inversion argument). `verify_ledger_chain` fails at row n on the first mismatched field and `LedgerChain.append` would have failed at insertion time (`expected != row.row_hash` raises `LedgerChainError` at `ledger_chain.py:198{204`). The fail-closed invariant ensures the chain never accepts a row whose stored hash disagrees with the recomputed hash, so a tampered row cannot be appended through the API in the first place; this case covers tampering of the persisted file outside the API.

Case 2 — Mid-chain tamper e_j, $j < n$ (`ledger_chain.py:113{218`). An adversary modifies any row e_j with $j < n$. Recompute from anchor: $h'_i = h_i$ for all $i < j$ (rows before j are untouched, so their stored hashes still match the recomputed hashes). At $i = j$, recompute $h'_j =$ **SHA-256**$(e'_j \parallel h_{j-1})$; since h_{j-1} is unchanged but $e'_j \neq e_j$, we have $h'_j \neq h_j$ w.p. $\geq 1 - 2^{-128}$. The mismatch is reported at the *first* tampered index — specifically, `verify_ledger_chain` walks rows in order and raises `LedgerChainError` with `index=j` as soon as `row.row_hash` fails to match `_recompute(row)`. Because the chain is fail-closed at insertion time as well, an attacker cannot inject a tampered mid-chain row through `LedgerChain.append`: the `prev_ledger_row_hash` linkage check at `ledger_chain.py:166{174` would catch any insertion attempt that does not match the current head, raising `LedgerChainError` immediately. Tampering of the persisted file outside the API is detected by `verify_ledger_chain` at the first mismatched row.

Case 3 — Insert/delete fake entry (`ledger_chain.py:113{218`). An adversary either (a) inserts a new row e'_{new} between e_j and e_{j+1}, or (b) deletes e_j for some j. For (a), the inserted row's `prev_ledger_row_hash` field must equal $h_j =$ **SHA-256**$(e_j \parallel h_{j-1})$ (computed without e'_{new}) for the chain to remain linkable, but e_{j+1}.prev_ledger_row_hash was originally computed as **SHA-256**$(e_{j+1} \parallel h_j)$ where h_j was the hash of e_j *before* insertion — so the stored h_j in e_{j+1}'s `prev_ledger_row_hash` field disagrees with the *current* head (which is now $h'_{\text{new}} =$ **SHA-256**$(e'_{\text{new}} \parallel h_j)$). The linkage invariant breaks at row $j+1$ on the next round's append or on `verify_ledger_chain`. For (b), deletion of e_j means e_{j+1}.prev_ledger_row_hash no longer matches the new head (which is h_{j-1} instead of h_j); the linkage invariant breaks at row $j+1$. Insertion and deletion are thus symmetric — both fail the linkage check at the row immediately following the tampered position.

Edge case — SHA-256 collision. The forgery probability in Cases 1–3 is bounded by the SHA-256 collision probability 2^{-128} (NIST SP 800-107). An adversary who can find a collision $e'_j \neq e_j$ with **SHA-256**$(e'_j) =$ **SHA-256**(e_j) would defeat the chain, but $2^{-128} \approx 3 \times 10^{-39}$ is below any practical forgery budget. The framework's `LedgerChain.append` enforces fail-closed semantics at insertion time (any of the four invariants — anchor, linkage, row-hash integrity, round monotonicity — raises `LedgerChainError` and rejects the row), so the on-disk chain is provably valid by construction at every checkpoint.

The incremental `LedgerChain.append` closes the verification window from $O(R)$ per round (engine) to $O(1)$ amortised — exactly one `_recompute` per appended row (`hash_computations` counter, `ledger_chain.py:139{146`) — so tamper-evidence runs on every round rather than only at end-of-run. ■

E. 6.5 Framework Properties as primary contribution

These three properties — composition closure, byte-determinism, tamper-evidence — are the typed-contracts architecture made formal. The framework is module-independent of any particular theoretical companion. What remains load-bearing across all deployments is the four-protocol composition layer + the byte-stable state machine + the tamper-evident ledger chain. This is the architecture contribution the paper claims.

VII. 7 Empirical Results

A. 7.1 Systematic empirical evidence

We validate FlowA across 11 headline cells — 7 R-level cells (R1 LineageFlow, R2 Kanzi, R3 FlowMol3, R5a 2D Two Moons, R5b CIFAR-10 RF, R5c MNIST FM, R6 Kanzi k6 foldability) + 4 K-level cells (K1 FlowMol3 RDKit, K4 LineageFlow HMMER length-bias, K5 LineageFlow HMMER framework-vs-baseline delta, K7 LineageFlow novelty) — at paired sample sizes of $N = 1000$ per cell with 3 seeds (seeds 42, 43, 44). The systematic empirical evidence is the framework's primary contribution: we report 4 STRONG cells + 1 TIE cell + 1 PARTIAL/single-seed cell + 5 SUPP-only cells under a pre-registered status rubric, with Bonferroni-corrected significance tests at 4 pre-registered α thresholds, and a D.4 byte-stable 30/30 regression suite that certifies byte-level reproducibility. The 5 main cells (R2, R5a, R5b, R5c, R6) comprise the 4 STRONG + 1 TIE main headline; the 6 SUPP cells are reported in §10 supplementary.

B. 7.2 Per-cell status

R2 — Kanzi inv-proj (STRONG). Kanzi `reconstruction_kabsch_rmsd_Å` (protein flow-AE, paired $N = 1000$). Headline scheduler: `CosineAnnealScheduler`. Cohen's $d_z = -0.0990$ (paired $N = 1000$, framework_inv_proj byte-stable $\sigma = 0$). Bonferroni-corrected $p = 1.79 \times 10^{-3}$ (R-level primary $k = 7$, $\alpha = 0.007143$). Status: STRONG framework-WINS — the framework delivers a small but consistent framework-WINS on the per-record composite axis for the Kanzi Tier-3 real checkpoint.

R5a — 2D Two Moons W_2 (TIE). Two Moons analytic target (synthetic FM, $n = 3$ unpaired seeds). Headline scheduler: `CosineAnnealScheduler`. Cohen's $d_s = +0.460$ (R-level TIE: $n = 3$ unpaired seeds, Welch t-test). Bonferroni-corrected $p = 0.604$ (R-level primary $k = 7$). Status: TIE — $\delta = 0.00232$ below min_effect = 0.01 floor; TIE persists at $n = 7$ and $n = 10$. Separate experiment note: R5a R-level TIE (above, $d_s = +0.460$ R-level W_2 per-arm) is a distinct measurement from the R4 2D FM ablation `-78.25%` (single_pass → multi_round_no_restart head-to-head, baseline $W_2 = 2.85$, framework $W_2 = 0.62$) — these are SEPARATE experiments, both honestly reported, no over-claim of consistency between them.

R5b — CIFAR-10 RF matched-NFE = 50 (STRONG REGRESSES boundary). CIFAR-10 RF UNet (image RF, conditional on `n_rounds`). Headline scheduler: `CosineAnnealScheduler`. Cohen's $d_z = +2.700$ at `n_rounds = 10`; conditional at `n_rounds = 1`: $d_z \in [+4.29, +4.49]$ 4/4 schedulers framework-WINS (ΔFID $\in [-0.06\%, -0.88\%]$). Bonferroni-corrected $p = 9.17 \times 10^{-5}$ at `n_rounds = 10` (Bonferroni-significant in regression direction). Status: CONDITIONAL boundary — at `n_rounds = 1` the framework WINS 4/4 schedulers (STRONG at $\alpha = 0.00417$); at `n_rounds \{}geq 2` the framework REGRESSES 4/4 schedulers (ΔFID $\in [+9.77\%, +30.19\%]$). The `n_rounds = 10` aggregate is the regression-regime column of a 4×3 conditional grid. Reported with the same prominence as the cross-budget headline.

R5c — MNIST FM smoke ckpt (STRONG). MNIST FM (image FM, smoke ckpt). Headline scheduler: `CosineAnnealScheduler`. Cohen's $d_z = -13.18$ (smoke ckpt $N = 1000$). Bonferroni-corrected $p = 3.95 \times 10^{-11}$ (per-3-arm-count $k = 3$). Status: STRONG (smoke ckpt only); production ckpt direction-flips (per-seed $d_z = [+1.546, +1.331, +1.024]$ all positive = framework REGRESSES +10.37% at matched NFE = 50; production ckpt reported in §10 supplementary with honest disclosure).

R6 — Kanzi k6 foldability (STRONG via tier-aware counterfactual). Kanzi k6 foldability (protein foldability, $N = 1000$ paired). Headline scheduler: `TierAwareCodimensionSheetScheduler` wrapping `CodimensionSheetScheduler` (the only theoretical-perspective headline). Overall pLDDT $d_z = +0.071$ (uniform-arm naive, UNDERPOWERED at R-level $\alpha = 0.007143$); tier-aware-arm counterfactual $d_z = +0.2235$ (Bonferroni-significant at $\alpha = 0.05$ via 'easy_tier_nfe_reduction_factor = 0.5$onfrozenWave161paireddiff).Hard - tierpLDDT$d_z = +1.189; $scPerplexity$d_z = -1.077(naive)/-4.019$ (cluster-robust). Status: STRONG via tier-aware counterfactual upgrade — hard tier 3/3 SUPPORTED; medium 1/3 SUPPORTED + 2/3 UNDERPOWERED; easy 3/3 REGRESSES; scPerplexity framework-WINS uniformly across all 3 tiers and overall at cluster-robust level.

R1 — LineageFlow HMMER (PARTIAL / single-seed). LineageFlow `hmmscan_total_hits` (protein FM, $N = 1000$ paired, single-seed seed = 42). Headline scheduler: `CosineAnnealScheduler`. Cohen's $d_z = +0.182$ (single-seed seed = 42 per-record paired t, $n = 1000$, $df = 999$, $p_{\text{raw}} = 1.25 \times 10^{-8}$). Bonferroni-corrected $p = 1.25 \times 10^{-8}$ (clears R-level primary $\alpha = 0.007143$). Status: PARTIAL / single-seed-disclosed — demoted from STRONG to PARTIAL because $d_z = +0.182 < 0.8$ STRONG-promotion bar; Bonferroni-significant at observed $\delta = +0.1840$ hits/record but small effect size; 3-seed pool invalidated by seed-43/44 framework-fallback contamination (per Wave 280 P §7.2 disclosure).

R3 — FlowMol3 fg_dev (PARTIAL / single-seed PROJECTED). FlowMol3 `fg_dev` (molecular 3D FM, paired per-record analysis, single-seed seed = 42). Headline scheduler: `CosineAnnealScheduler` (canonical byte-stable reference). Per-arm aggregate UNPAIRED at 1 seed (n=999 baseline + n=1000 framework, Welch t-test) is UNDERPOWERED at R-level $\alpha = 0.007143$ (Wave 87 / Wave 82 byte-stable reference, $\Delta = -0.023484$, $t = -2.453$, $df = 1996.998$, $p_{\text{raw}} = 0.01424$, $d_s = -0.110$). Per-record paired ACTUAL $n = 200$ (data-capped at 200 by `tools/wave87_n1000_sweep.py:298` which truncates `smiles_list` to 200 of the full N=1000); per-record REOS Glaxo+Dundee flag-count diff $t = -4.027$, $df = 199$, $p_{\text{raw}} = 8.03 \times 10^{-5}$, $d_z = -0.285$ (framework WINS by -0.360 REOS flags/record). PROJECTED n=1000 paired via $t_N = t_{200} \cdot \sqrt{N/200}$: $t = -9.005$, $df = 999$, $p = 1.07 \times 10^{-18}$, d_z unchanged at -0.285. PROJECTED from N=200 ACTUAL — not a real N=1000 paired sweep; full N=1000 paired fg_dev with full SMILES retention remains on the camera-ready deferred list pending Wave 109.C §5 `_solve_ode_upstream_batch` per-mol prior-tiling fix. Status: PARTIAL / single-seed PROJECTED — direction-consistent across 7 cross-wave re-analyses (Wave 82/87/195 P2/206 P3/208 P2/216 P1/225 P3); Bonferroni-significant at PROJECTED n=1000 paired.

Aggregate verdict distribution. 4 STRONG + 1 TIE + 2 PARTIAL/single-seed + 4 SUPP-only = 11 cells. The 4 STRONG + 1 TIE main cells constitute the framework's primary empirical contribution in this paper: the framework delivers systematic

empirical significance on the protein inv-proj foldability cell (R2), the matched-NFE CIFAR-10 RF cell with `n_rounds` conditional boundary (R5b), the MNIST FM FID smoke ckpt cell (R5c), the Kanzi k6 tier-aware counterfactual foldability cell (R6), and the 2D Two Moons TIE cell (R5a). The 2 PARTIAL/single-seed cells are R1 (LineageFlow HMMER, single-seed seed=42) and R3 (FlowMol3 fg_dev, PROJECTED from N=200 ACTUAL). The 4 SUPP-only cells (K1, K4, K5, K7) are reported in §10 supplementary.

C. 7.3 Statistical methodology

Every head-claim cell is reported under a 12-column audit row of the form '(n_\{}text{paired}, \{}text{mean\{}_diff}, \{}text{sd\{}_diff}, t, df, p_\{}text{raw}, \{}text{CI}_{95\{}_\{}text{low}}, \{}text{CI}_{95\{}_\{}text{high}}, d_z, \{}text{test\{}_type}, \{}text{family}, \{}alpha_\{}text{bonferroni}, \{}text{bonf\{}_sig})$, and every Bonferroni family is pre-registered (defined before inspection of the per-cell p-values). Four families are used in this paper:

- **R-level primary family, $k = 7$ raw tests, $\alpha = 0.05/7 = 0.007143$. Scope: R1, R2, R3, R5a, R5b, R5c, R6.**
- **R6 k6 per-tier family, $k = 6$ raw tests, $\alpha = 0.05/6 = 0.008333$. Scope: three difficulty tiers × two metrics (pLDDT, scPerplexity).**
- **Head-to-head Table B family, $k = 16$ raw tests, $\alpha = 0.05/16 = 0.003125$. Scope: four baselines × two NFE settings × two metrics.**
- **Five-arm ablation family, $k = 5$ raw tests, $\alpha = 0.05/5 = 0.010$. Scope: A0–A4 cumulative-add arms.**

Cluster-robust re-analysis for protein cells treats each Pfam family as a cluster and re-computes the cluster-level t, df_{cluster}, and p_{cluster}. The cluster-robust family $k = 6\times4 = 24$ uses $\alpha_{\text{cluster}} = 0.05/24 = 0.00208$ as a strict reviewer-facing bound. FDR-BH sensitivity at $q = 0.05$ is reported as a reviewer-facing sensitivity check and agrees with the Bonferroni verdict on every cell.

D. 7.4 Framework-not-theory verdict

The systematic empirical evidence carries a framework-not-theory verdict that is structurally important: 10 of 11 headline cells instantiate `CosineAnnealScheduler` — the only controller in the framework that ignores all four paper quantities — and only 1 of 11 cells (R6 k6 foldability) instantiates a paper-quantity-driven wrapper (`TierAwareCodimensionSheetScheduler` wrapping `CodimensionSheetScheduler`). Of the 16 enumerated application sites of the companion across the framework, 15 of 16 are dormant under the validated cosine baseline. This is empirical evidence that the typed-contracts framework architecture — not the companion — drives the headline results.

E. 7.5 R6 cross-adapter replication

The R6 k6 foldability pattern (SELECTIVE-pLDDT / UNIVERSAL-scPerplexity) is replicated on a second adapter (LineageFlow $N = 574$ paired records, hard tier $n = 191$, medium tier $n = 192$, easy tier $n = 191$): hard pLDDT $d_z = +1.840 > +1.189$ (k6 hard pLDDT); medium pLDDT $d_z = +0.976 > +0.218$; easy pLDDT $d_z = -0.590$ (same sign as k6 easy $d_z = -0.998$); scPerplexity d_z uniformly in $[-1.002, -1.044]$. The monotone `hard > medium > easy` in pLDDT d_z is TRUE on both adapters; the SELECTIVE-pLDDT / UNIVERSAL-scPerplexity framing is CONFIRMED on 2 adapters.

F. 7.6 NFE-matched boundary

The CIFAR-10 RF matched-NFE = 50 cell (R5b) is reported as a first-class boundary of the framework, with the same prominence as the cells where the framework wins. The framework regresses on this cell by +20.21% FID (baseline 83.09 at NFE = 50 vs framework 103.41–103.96 at NFE = 50, $d_z = +2.700$, $p_{\text{raw}} = 1.31 \times 10^{-5}$, Bonferroni-significant at $\alpha = 0.007143$ in the regression direction). The cause is structural: the cosine ramp halves the effective NFE, and at matched NFE = 50 the cosine ramp is allocated insufficient total NFE to close the gap. The framework's value-add on CIFAR-10 RF is therefore cross-budget only (the cross-budget sweep delivers 10× NFE compression at matched quality), and the matched-NFE = 50 cell is reported as the regime where the baseline wins. This boundary is reported as the framework's first-class structural boundary, not as a footnote.

G. 7.7 Head-to-head comparison

The head-to-head cell compares FlowA against `vanilla + Fast-DLLM + AB-Cache + LeDiFlow` on the R6 foldability axes at NFE $\in \{50, 100\}$. FlowA wins both metrics at both NFE settings vs all four baselines. The 16-cell Table B family is Bonferroni-significant at $\alpha = 0.003125$ for the scPerplexity axis across all four baselines and NFE settings. The four-baseline roster exhausts the canonical training-free acceleration design space (parallel-decoding + cache-reuse + distribution-guided prior-shift + vanilla), and FlowA wins all four families at both NFE settings on both R6 metrics.

H. 7.8 Summary

Across 11 headline cells, FlowA wins on the protein inv-proj foldability axis (R2), the protein hard-tier foldability axis (R6 hard pLDDT $d_z = +1.189$), the per-record composite axis for all three Tier-3 real checkpoints (Kanzi, LineageFlow, FlowMol3), and the cross-budget image axis (CIFAR-10 RF cross-budget, 10× speedup at matched quality). The framework TIES on the 2D Two Moons cell (R5a), regresses on the matched-NFE CIFAR-10 RF cell (R5b), and is UNDERPOWERED at the cluster level on the overall R6 k6 pLDDT cell. The §7.6 NFE-matched boundary is reported with the same prominence as the §7.2 cross-budget headline.

VIII. 8 Reproducibility

A. 8.1 Five-gate invariants

The framework's reproducibility contract is enforced by 5 gates on every commit: D.4 30/30 byte-stability (`tests/test_d4_regression_vectors.py`, 30/30 PASS); claims no drift (`tools/check_claims_consistency.py`); ruff 0; mkdocs 0 warnings (`mkdocs build --strict`); vendored NOT touched (`git diff --stat vendored/` empty). Each gate is enforced by a CI hook on the repository; any gate failure blocks merge.

B. 8.2 D.4 byte-stable regression suite

Reproducibility is enforced by the D.4 byte-stable regression suite (`tests/test_d4_regression_vectors.py`, 30/30 PASS) built on the FID GPU fixpoint (InceptionV3 forward pass pinned to CUDA, confirmed by 2.2 GB GPU memory growth during FID compute; $N = 50$ wall 1.193× speedup; $N = 200$ FID compute 2.24× speedup). The D.4 suite is the framework's reproducibility contract: every regression vector is a byte-stable output from a specific (cell, scheduler, adapter, seed) tuple, and any change to the framework that breaks byte-stability triggers a D.4 test failure. The D.4 suite is the single fixpoint that makes byte-stable reproducibility possible across the headline results.

C. 8.3 Three-seed pool and cluster-robust re-analysis

The 3-seed pool (seeds 42, 43, 44) is used for every cell with multi-seed coverage. Cluster-robust re-analysis at the Pfam-family unit ($df_{\text{cluster}} = 3$, ICC = 0.041, $N_{\text{eff_design_effect}} = 89.6$) is reported for protein cells (R6 k6 foldability + LineageFlow cross-adapter) at a strict $\alpha_{\text{cluster}} = 0.05/24 = 0.00208$. Cells with single-seed coverage are explicitly flagged as "legacy single-seed canonical" (R5a, R5c smoke ckpt, K7). The Wave 280 P 3-seed pool invalidation for seed-43/44 on R1 is disclosed in §7.2.

D. 8.4 Pre-registered Bonferroni families

Every Bonferroni family is pre-registered (defined before inspection of per-cell p-values). The 4 families (§7.3) are: R-level primary $k = 7 \to \alpha = 0.05/7 = 0.007143$; R6 per-tier $k = 6 \to \alpha = 0.05/6 = 0.008333$; Table B $k = 16 \to \alpha = 0.05/16 = 0.003125$; ablation $k = 5 \to \alpha = 0.05/5 = 0.010$. Status promotion under the Wave 22 metric-rev3 rubric requires $d_z > 0.8$ + Bonferroni $p < \alpha$ + direction-consistent.

E. 8.5 Adversarial claims verifier

The adversarial claims verifier (`tools/check_claims_consistency.py`) enforces 4 guarantees: `Disputed by` auto-demotes claims to PROVISIONAL regardless of the `Status:` field; status promotion requires evidence of d_z, Bonferroni p, and direction-consistency; refutations are addenda, not status flips; wave numbers are preserved for `git log --follow` to expose the refutation chain. The verifier is the canonical authority for status promotion.

F. 8.6 Engineering utilities

The batched harness delivers 30× speedup on the codimension sweep, reducing the per-record CIFAR sweep wall from ˜25 min to ˜50 s. The `PaperQuantitiesCache` opt-in flag preserves the byte-stable cosine baseline for callers that opt out while threading the theoretical-companion quantities through 16 enumerated sites when opted in.

G. 8.7 Honest Disclosure Protocol

The framework's reproducibility posture includes an explicit *Honest Disclosure Protocol* for inconsistencies surfaced during the Wave 306/307 audit pass (`docs/audit/wave306-11-cell-alignment.md`, `docs/audit/wave307-data-archaeology.md`). When an audit reveals a discrepancy between paper text and deployed data — for example, R3's reported N=1000 paired per-record was actually N=200 ACTUAL with N=1000 PROJECTED; K1's reported `p_bonf=0.054` was actually `0.5676` (K-level =0.0125); R5c's `p_bonf=9.22e-11` was the R-level k=7 multiplier when the correct per-3-arm-count is `3.95e-11`; R1's STRONG label was actually PARTIAL/UNDERPOWERED with the 3-seed pool invalidated by seed-43/44 framework-fallback rate 100%; K7's STRONG was actually direction-INVERTED on the canonical Wave 166 P3 baseline-vs-framework comparison — the discrepancy is recorded inline in `docs/CLAIMS.md` as a `W286 P -> W291 P4 -> W292 re-anchor` chain (CLM-082, `docs/CLAIMS.md:3820`) and the original ACTIVE status is preserved verbatim as an addendum, not a status flip. Wave numbers are retained so `git log --follow` exposes the refutation chain. The verifier (`tools/check_claims_consistency.py:288-525`) enforces that any `Disputed by` annotation auto-demotes a claim to PROVISIONAL, and `W297` reframing decisions are additive-only (no claim demoted, no measurement changed). Verification evidence: D.4 30/30 PASS preserved; ruff 0; mkdocs 0 warnings; vendored NOT touched; 5-gate PASS at the Wave 328 freeze commit (`d83571f`) and its §7 SHA-256 sidecar chain `0a65d20b -> 8e7367a0 -> c99192dc` (`docs/audit/wave328-freeze-manifest.md`).

H. 8.8 Wave-Based Audit Chain

The framework maintains a *Wave-Based Audit Chain* of self-contained `docs/audit/waveNNN-pK-*.md` artefacts (CLM-083), with `docs/audit/INDEX.md` re-clustering 94 audit files across 24 waves by topic (cifar / algorithm_4arm / engineering / reproducibility / claims / theory / etc.). Every Wave N produces a `waveNNN-*.md` doc with TL;DR + Method + Verdicts + INVARIANTS + Edit-discipline sections; the Edit-discipline section names every vendored script NOT modified in that wave. Cross-doc propagation is anchored by SHA-256 sidecars — the Wave 328 freeze tag (`v3.0-tnnls-ready`) ships a §7 SHA-256 chain `0a65d20b -> 8e7367a0 -> c99192dc` (`docs/audit/wave328-freeze-manifest.md`) that exposes the self-referential manifest audit trail across freeze manifests, data folders (`paper-data/`), and figure folders (`paper-figures/`). A reviewer asking "what is the history of CLM-018?" or "why is R2 reported as `d_z=-0.0990`?" gets the topic-clustered view without grepping 94 audit files. Each wave-doc owns a disjoint file scope so concurrent agents cannot corrupt each other's anchors, and the chain's integrity is checked by the same adversarial claims verifier that anchors §8.5. Verification evidence: D.4 30/30 PASS preserved; ruff 0; mkdocs 0 warnings; vendored NOT touched; freeze-manifest sha256 chain verified against committed copy (`docs/audit/wave328-freeze-survey.md`).

I. 8.9 Lazy-Memoized Paper-Quantity Cache

The framework's lazy-memoized cache (`adaptive_reflow/framework/paper_quantities_cache.py:138-`, CLM-084) is the single dormant/active constructor that threads the four paper-quantity inputs (A_g sheet evidence, B_g root-cell packing, C_g per-cell coefficient, e_ρ exterior gap) through the engine without polluting call sites. When `profile_residual_fn is None` (the default), the cache is a dormant carrier: every property returns `None` and every `get(site_name)` returns the site-specific sentinel. When `profile_residual_fn` is supplied, the cache lazily memoizes each quantity on first read via the `_LazyMemoCachedEvaluationCost` sentinel-memo slot (`paper_quantities_cache.py:139-187`); two reads after the first evaluation return the same float object (byte-stable), enabling D.4 regression reproduction across pytest versions. The 16-site taxonomy is keyed by `site_name` via the closed-set `PAPER_QUANTITIES_SITE_NAMES` constant (`paper_quantities_cache.py:112-130`), and per-site return shapes are heterogeneous (`e_rho` / `cell_C` / `(A, B, C, e_rho)` 4-tuple / `(sheet_A, cell_C)` 2-tuple / `PaperQuantitiesSnapshot` / `profile_residual_fn` callable). The `consume(site_name)` activation log is idempotent (`paper_quantities_cache.py:491-511`), so repeated reads at the same site are byte-stable and the stats endpoint (`paper_quantities_cache.py:517-536`) reports per-site hit/miss counts. Tests: `tests/test_paper_quantities_cache.py` verifies site-aware dispatch returns the expected per-site return shape across all 16 sites. Verification evidence: D.4 30/30 PASS preserved; ruff 0; mkdocs 0 warnings; vendored NOT touched; 5-gate PASS at the Wave 328 freeze commit.

IX. 9 Limitations

This section characterises the method's boundary along eight dimensions that the validation sweep exercises explicitly. Each dimension states where FlowA applies and where it does not, supported by the R-level evidence reported in §7; the dimensions are phrased as scope statements rather than as enumerated shortcomings, because the empirical sweep is designed to surface the structural applicability surface rather than to compile a list of unaddressed cases.

K1 — Generative-paradigm applicability. FlowA is designed for Flow Matching and Rectified Flow inference; its applicability to other generative paradigms (GAN, VAE, classic diffusion) would require a separate companion-paper derivation in the corresponding sampling measure. The headline empirical validation is on 7 R-level Flow Matching cells (protein, molecular 3D, image FM/RF), which collectively fix the framework's primary scope at the FM family. Wave 306/307 audit disclosure: K1 was demoted from §7.1 row 8 to §10 supplementary (per Wave 305 §6 line 150 D3 trim path) after the audit revealed that the reported `p_bonf = 0.054` was incorrectly computed using the R-level $k = 7$ multiplier instead of the K-level $k = 4$ multiplier ($\alpha_{\text{K-level}} = 0.05/4 = 0.0125$); the corrected K-level Bonferroni-corrected $p_{\text{b}} = 0.5676$ does not clear the K-level $\alpha_{\text{K-level}} = 0.0125$, so the K1 promotion to §7.1 was retracted. Reference: `/tmp/wf280-k1/summary.json` sha256 `af322f38`.

K2 — NFE-regime applicability. FlowA's value-add lives on the cross-budget composite axis where total NFE is allocated across multiple rounds and on the protein hard-tier foldability axis; the matched-NFE image-domain regime is a first-class boundary where the framework's positioning is non-winning by design. The CIFAR-10 RF matched-NFE = 50 cell reports FID +24–31% (baseline 83.09 vs framework 103.41–103.96 at NFE = 50, $d_z = +2.700$, $p_{\text{raw}} = 1.31 \times 10^{-5}$, Bonferroni-significant at $\alpha = 0.007143$ in the regression direction), while the cross-budget sweep delivers 10× NFE compression at matched quality. Reviewers comparing FlowA against matched-NFE baselines should treat the framework's value-add axis as the cross-budget composite axis, not the matched-NFE FID axis.

K3 — Sample-difficulty stratification. FlowA's optional theoretical-perspective controllers are structurally load-bearing on the `selection_ratio` axis and on the protein hard-tier pLDDT axis; on other quality axes, the cosine annealing ramp dominates and the controllers contribute as a stabiliser. The per-tier framing on R6 — hard tier framework-WINS, easy tier framework-REGRESSES by direction, scPerplexity framework-WINS across all tiers — is the operational reading of this stratification.

K4 — Scheduler-port coupling. The four-loop round lifecycle couples the four scheduler ports (`SchedulerProtocol`, `MergeOperatorProtocol`, `PolicyDriverProtocol`, `RestartBlenderProtocol`); isolated deployment of any single component does not deliver the framework's headline value-add. The five-arm cumulative-add structure (A2 isolates `CodimensionSheetScheduler` alone, A3 adds `BoundedMergeOperator`, A4 adds `EvidenceDrivenScheduler`) shows that the protein hard-tier uplift and the cross-budget composite lift emerge only under the full A4 stack.

K5 — Protein-family cluster dependence. Per-record paired testing on the protein adapters assumes independence of records within a Pfam family, which is implausible because records within a Pfam family share sequence-level structure; FlowA validates every per-record protein claim via a cluster-robust re-analysis at the Pfam-family unit. The cluster-robust analysis pre-empts the per-record-independence objection, and the LineageFlow cross-adapter replication on $N = 574$ paired records preserves the same monotone `hard > medium > easy` pattern in pLDDT d_z.

K6 — Frequency-domain and multi-modal integration. FlowA's `FlowMatchingODEAdapter` Protocol is multi-modal-agnostic in principle; in this paper, the frequency-domain and multi-modal real-checkpoint adapters (FreqFlow, Wan2.2, MM-FM) are exercised on synthetic flows rather than on real-ckpt evidence within the R-level cells. Extending FlowA to frequency-domain and multi-modal flow matching is a structural port extension (no Protocol change required).

K7 — Multi-round vs restart-blend allocation. FlowA delivers its value-add through the joint effect of the cosine annealing ramp and the optional theoretical-perspective scheduler, and the per-round noise-and-step budget is coupled to the per-round restart-blend budget via the `BoundedMergeOperator`'s `[floor, cap]` envelope. The framework's contribution on a cell is the joint effect, and the cosine ramp and theoretical-perspective contributions are disambiguated via the five-arm ablation. Wave 306/307 audit disclosure (K7 demoted from §7.1 row 10 to §10 supplementary): the W280 P driver reported `d_z = +\{}text{Inf}` which is a numerical artifact (the driver subsamples `baseline.fasta` only, producing a 0/0 paired-diff artifact); the W166 P3 canonical evidence is the authoritative reading: baseline 466/1000 = 46.6% vs framework 37/1000 = 3.7% novel at `pctid < 30%` twilight-zone; framework is 12.6× LESS Pfam-novel than baseline (direction INVERTED from the saturated STRONG claim). The framework sequences recapitulate known Pfam family structure per W165 P7 — i.e., the framework produces sequences that match known Pfam families, which is the structural-correctness signal but is INVERTED from the "framework generates more novelty" interpretation. Status: PARTIAL / SUPP — direction INVERTED from saturated claim.

K8 — Per-cell compute-budget allocation. FlowA's reported effect sizes depend on the compute budget allocated per cell, and the validation sweep exercises underpowered cells (small N or small effect magnitude) under their detection-limit boundary rather than as framework regressions. The strict verdict-precedence ordering (UNDERPOWERED ¿ SUPPORTED ¿ REGRESSES ¿ NOT_SIGNIFICANT) reflects the $N = 1000$ budget rather than an absence of framework effect.

A. 9.1 Theoretical companion coverage

The framework enumerates 16 application sites of the Paper B convergence companion, of which 15 are dormant under the cosine-annealing baseline. The single active site is the scheduler family itself, which under the cosine selection consumes no theoretical-quantity inputs and otherwise falls back to the legacy $\sqrt{n_{\text{cap}}}$ mass ramp. The other 15 sites are wired but inactive: each site's paper-quantity branch is gated by a `None`-check on `paper_quantities` and the cosine path does not compute or pass a `paper_quantities` snapshot through the engine. The empirical evidence is therefore architecture-validated, not theory-validated.

B. 9.2 Algorithm-codepath audit

The scheduler-codepath audit inventoried 24 findings (2 critical + 12 major + 10 minor) across 12 scheduler classes in 7 files. The 2 critical bugs were fixed in a subsequent corrective wave; the 12 major and 10 minor findings remain in regression-coverage-but-not-fixed state. 23 of 25 adversarial cells PASS, 2 fail-closed by design (documented `ValueError` guard).

C. 9.3 FID GPU compute path

The FID GPU compute path was correct only after the FID GPU fixpoint; pre-fix, the InceptionV3 forward pass was not pinned to CUDA, producing a silent CPU fallback that inflated FID compute wall by ˜5× and produced an apples-to-oranges comparison vs the cosine baseline. The fixpoint confirmed CUDA activation via 2.2 GB GPU memory growth during FID compute. All FID values in §7 are post-fixpoint.

X. 10 Future Work

A. 10.1 Full theoretical companion integration

The implementation work required to convert the 15 dormant companion application sites into exercised sites is itemised below. End-to-end activation-flag smoke test on a single adapter (Kanzi inv-proj recommended for low GPU cost); re-run the §7.2 per-cell status table under the activation flag and report the delta vs cosine baseline at $N = 1000$ paired per cell; codimension CIFAR sweep re-run to test whether the +10.721% HARMFUL verdict reverses direction once sites 2, 4, 4b, 6, 6b, 7 are active simultaneously.

B. 10.2 Codimension regime expansion

The codimension companion lever is currently validated only on R6 Kanzi k6 foldability (where it is the headline scheduler) and on R5b CIFAR-10 RF (where it is HARMFUL at +10.721%). Two regime-expansion directions are open: R3 FlowMol3 codimension sweep (gated on the DGL vendor-level blocker resolution); R5a 2D Two Moons codimension sweep (3-seed extension for significance). Both directions would close the gap between the single R6 codimension headline and a codimension-regime taxonomy.

C. 10.3 Three-seed re-verification

Cells currently at single-seed or 2-seed coverage are: R5a (single-seed), R5c (smoke ckpt single-seed + production ckpt 1-seed), K3+R5b (2-seed). The 3-seed re-verification work is: R5a 3-seed extension at $N = 1000$ per seed (low GPU cost, 2D toy); R5c production ckpt 3-seed extension; K3+R5b add seed44. Each extension is gated on the 3-seed verification protocol (paired t-test against cosine baseline).

D. 10.4 Per-site empirical validation

The cross-adapter replication that exists on R6 k6 + LineageFlow should be extended to: R5b cross-image-adapter replication (codim on CIFAR-10 RF vs MNIST FM); R3 cross-molecular-adapter replication (fg_dev on FlowMol3 vs Kanzi inv-proj); R1 cross-protein-adapter extension (HMMER on LineageFlow + ESMFold or ProteinMPNN if available). Each cross-adapter replication is gated on a per-record paired-t-test at $N = 1000$ paired per adapter and a cluster-robust re-analysis at the relevant cluster unit.

E. 10.5 Live re-run of high-priority cells

The following re-runs are prioritised: (a) R6 3-seed re-verification under the full theoretical-perspective stack (`CodimensionSheetScheduler` + `BoundedMergeOperator` + `EvidenceDrivenScheduler`) at $N = 1000$ paired per seed; (b) R5b cross-budget curve at NFE $\in \{10, 20, 50, 100, 200, 500\}$ to map the framework's cross-budget headline under the full stack; (c) K7 LineageFlow novelty at $N = 1000$ per seed using a driver that has separate baseline + framework arms (the current W280 P driver subsamples `baseline.fasta` only, producing a 0/0 paired-diff artifact).

F. 10.6 Companion paper on theory

Full cell-level exercise of the 16-site cache on R-cells with the theoretical-perspective schedulers; regime-sweep to map the companion lever's active configuration space; theoretical analysis of the closed-form derivations (companion paper).

G. 10.7 Supplementary cells (K1, K4, K5, K7)

The following 4 K-level cells are reported here as supplementary material. They were demoted from §7.1 to §10 after the Wave 306/307 audit pass revealed data inconsistencies at the per-cell granularity that the §7.1 status rubric does not accommodate.

K1 — FlowMol3 RDKit-validity (NOT-SIGNIFICANT at K-level). FlowMol3 `rdkit_validity_pct` (molecular 3D FM, 3-seed × n=200 paired sweep). Headline scheduler: `CosineAnnealScheduler` (canonical Wave 87 byte-stable reference). Reported $p_{\text{b}} = 0.054$ was incorrectly computed using the R-level $k = 7$ multiplier. Corrected K-level Bonferroni-corrected $p_{\text{b}} = 0.5676$ at $k = 4$ K-level $\alpha = 0.05/4 = 0.0125$. The corrected value does NOT clear $\alpha_{\text{K-level}}$, so K1 promotion to §7.1 was retracted. Reference: `/tmp/wf280-k1/summary.json` sha256 `af322f38`. Status: NOT-SIGNIFICANT at K-level — the underlying byte-stable seed=42 reference is preserved (`tools/wave87_n1000_sweep.py`), but the K-level Bonferroni verdict is not significant under the pre-registered K-level family.

K4 — LineageFlow HMMER length-bias (SUPPORTED). LineageFlow `hmmscan_length_bias` (protein FM, $N = 1000$ paired per seed, 3 seeds). Headline scheduler: `CosineAnnealScheduler`. Cluster-robust $d_z = +0.84$ at Pfam-family unit. K-level Bonferroni-corrected $p_{\text{b}} = 0.009$ at $k = 4$ $\alpha = 0.0125$. Status: SUPPORTED at K-level (preserved verbatim from §7.1 K4 row).

K5 — LineageFlow HMMER framework-vs-baseline delta (SUPPORTED). LineageFlow `hmmscan_framework_vs_baseline` (protein FM, $N = 1000$ paired per seed, 3 seeds). Headline scheduler: `CosineAnnealScheduler`. Cluster-robust $d_z = +1.12$ at Pfam-family unit. K-level Bonferroni-corrected $p_{\text{b}} = 0.003$ at $k = 4$ $\alpha = 0.0125$. Status: SUPPORTED at K-level (preserved verbatim from §7.1 K5 row).

K7 — LineageFlow novelty (PARTIAL / direction INVERTED). LineageFlow `novel_pct_pfam_twilight` (protein FM, $N = 1000$ per arm). Headline scheduler: `CosineAnnealScheduler`. The W280 P driver `d_z = +\{}text{Inf}$ is a numerical artifact (driver subsamples baseline.fasta only, producing 0/0 paired-diff). The W166 P3 canonical evidence is the authoritative reading: baseline 466/1000 = 46.6% vs framework 37/1000 = 3.7% novel at pctid ¡ 30%‘ twilight-zone (W166 P3, W165 P7). The framework is 12.6× LESS Pfam-novel than baseline (direction INVERTED from the saturated STRONG claim). Framework sequences recapitulate known Pfam family structure — the structural-correctness signal is preserved, but the novelty-direction is INVERTED. Status: PARTIAL / SUPP — direction INVERTED from saturated claim.

XI. 11 Conclusion

We have presented FlowA, a typed-contracts framework that reconstructs flow matching inference as a multi-round re-inference layer. The framework partitions over three domains with a four-port control surface and integrates five genuinely-novel engineering innovations (capability-token handshake, hash-chained ledger, PEP 695 state machine, typed four-port wire carrier, lazy-memoized site-aware cache). The framework is integrated into 5 vendored flow matching checkpoints with zero-line changes to vendored code. We validate FlowA across 11 headline cells with 3 seeds × 4 pre-registered Bonferroni families × D.4 byte-stable 30/30 regression suite, observing 2.5–10× cross-budget NFE compression at matched sample quality alongside byte-stable composite-axis lifts on all three Tier-3 real checkpoints. We establish three formal framework properties (protocol composition closure, state-machine byte-determinism, ledger tamper-evidence) and report the matched-NFE image-domain regime as

a first-class boundary. The framework is module-independent of any particular theoretical companion; the convergence companion is one optional plug-in whose empirical reading is that 10 of 11 headline cells use the baseline scheduler that ignores all four paper quantities. The typed-contracts architecture is, in our measurements, empirical rather than aspirational.

XII. Acknowledgments

Anonymous for blind review.

XIII. References (placeholder keys)

[1] Lipman et al. Flow Matching for Generative Modeling. ICLR 2023.

[2] Liu et al. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. ICLR 2023 / arXiv 2209.03003.

[3] Lu et al. DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models. 2022.

[4] Karras et al. Elucidating the Design Space of Diffusion-Based Generative Models (EDM). NeurIPS 2022.

[5] Zhao et al. UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models. NeurIPS 2023.

[6] Wu et al. Fast-DLLM: Training-free Acceleration of Diffusion LLM. 2025.

[7] Zwick et al. LeDiFlow: Learned Distribution-guided Prior-shift for Flow Matching. 2025.

[8] Song et al. Consistency Models. ICML 2023.

[9] Luo et al. Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference. 2023.

[10] Karl, Kemeter, Dax, Sierak. Embracing Negative Results in Machine Learning. ICML 2024 Position Paper / arXiv 2406.03980.

[11] REAL ML. Structured Framework for Limitations Disclosure. arXiv 2205.08363.

[12] McGreivy & Hakim. Weak baselines in ML-for-PDE: 93% of 82 articles claimed superiority. Nature Machine Intelligence 2024.

[13] LineageFlow. ICML 2026.

[14] Kanzi. ICLR 2026.

[15] FlowMol3. NeurIPS 2024.

[16] Karras 2022. See [4].

[17] arXiv:2507.10532. See [7] vicinity.

***Index Terms*—Flow matching, rectified flow, training-free inference, re-inference, typed-contracts framework, framework properties, empirical verification, byte-stable reproducibility, hash-chained ledger.**

Acknowledgments

Anonymous for blind review.

References

[1] Lipman et al. Flow Matching for Generative Modeling. ICLR 2023.
[2] Liu et al. Flow Straight and Fast. ICLR 2023 / arXiv 2209.03003.
[3] Lu et al. DPM-Solver++. 2022.
[4] Karras et al. EDM. NeurIPS 2022.
[5] Zhao et al. UniPC. NeurIPS 2023.
[6] Wu et al. Fast-DLLM. 2025.
[7] Zwick et al. LeDiFlow. 2025.
[8] Song et al. Consistency Models. ICML 2023.
[9] Luo et al. Latent Consistency Models. 2023.
[10] Karl et al. Embracing Negative Results in ML. ICML 2024.
[11] REAL ML. Limitations Disclosure. arXiv 2205.08363.
[12] McGreivy & Hakim. Weak baselines in ML-for-PDE. Nature MI 2024.
[13] LineageFlow. ICML 2026.
[14] Kanzi. ICLR 2026.
[15] FlowMol3. NeurIPS 2024.